



Стратегии взаимодействия человека и ИИ при работе с кодом

Когда мы используем ИИ в разработке, важно понимать не только то, что он умеет генерировать код, но и какую роль мы ему отводим внутри рабочего процесса. Один и тот же инструмент может использоваться совершенно по-разному: от простой подсказки разработчику до самостоятельного выполнения целой цепочки действий.

Главный вопрос заключается не в том, может ли ИИ написать код. Современные модели уже способны создавать функции, объяснять ошибки и предлагать изменения. Гораздо важнее определить, где должен находиться человек в этом процессе и какой уровень самостоятельности мы готовы предоставить системе.

В разработке обычно выделяют три основных подхода к взаимодействию: человек постоянно контролирует работу ИИ во время выполнения задачи, человек проверяет результат после выполнения, либо ИИ получает возможность самостоятельно выполнять действия в заданных ограничениях.

Стратегия «человек в контуре»

Подход «человек в контуре» означает, что человек остаётся активным участником каждого важного шага работы.

В таком режиме разработчик ставит задачу, получает предложение от ИИ, анализирует его, при необходимости изменяет запрос, корректирует результат и только после этого использует полученный код в проекте.

Например, разработчик создаёт новую функцию. Он описывает требование, получает вариант реализации, изучает предложенный код, проверяет соответствие архитектуре проекта и самостоятельно принимает решение о дальнейшем использовании.

Главная особенность этого подхода заключается в постоянном взаимодействии. ИИ не выполняет задачу вместо разработчика, а выступает инструментом ускорения. Ответственность за итоговое решение остаётся у человека.

Такой вариант особенно подходит для ситуаций, где цена ошибки высокая или требуется глубокое понимание бизнес-логики. Например, при изменении критических частей системы, работе с безопасностью или изменении структуры данных.

Недостаток этого подхода заключается в том, что разработчик всё равно должен участвовать практически на каждом этапе. Если задача большая и состоит из множества повторяющихся действий, постоянный контроль может занимать значительное количество времени.

Стратегия «человек на ревью»

Следующий подход предполагает, что ИИ получает больше самостоятельности, а человек подключается уже после выполнения основной работы.

В этом случае разработчик формулирует задачу, система подготавливает результат, а человек выполняет проверку перед использованием.

Например, ИИ может подготовить новый модуль, изменить несколько файлов или предложить набор тестов. После этого разработчик изучает изменения, проверяет корректность и решает, можно ли их принять.

Такой подход похож на работу с другим разработчиком в команде. Мы не контролируем каждую написанную строку во время процесса, но обязательно выполняем проверку результата.

Преимущество этого режима заключается в скорости. Человек не тратит время на каждое небольшое действие, а концентрируется на анализе итогового результата.

Однако здесь повышается требование к качеству проверки. Если разработчик просто принимает результат без анализа, ошибки ИИ могут попасть в рабочую систему.

Поэтому в таком режиме особенно важны автоматические тесты, правила качества кода и понимание того, какие части результата требуют наиболее внимательной проверки.

Стратегия «автономный агент с лимитами»

Более высокий уровень самостоятельности представляет подход с автономным агентом.

Здесь ИИ получает не отдельную команду вроде «напиши функцию», а цель, которую необходимо достигнуть. Для выполнения задачи агент может самостоятельно определить последовательность действий, использовать доступные инструменты, анализировать промежуточные результаты и корректировать свои действия.

Например, задача может звучать как: подготовить изменения для обновления зависимости в проекте. Агент может изучить файлы, определить места использования старой версии библиотеки, предложить изменения, выполнить проверки и сформировать отчёт о результате.

Но автономность не означает полную свободу действий.

В реальной разработке агент всегда должен работать с ограничениями. Ему могут ограничивать доступ к файлам, запрещать изменение критических компонентов, устанавливать лимиты на выполнение команд или требовать подтверждения перед важными операциями.

Это необходимо потому, что ошибка автономной системы может иметь больший масштаб, чем ошибка отдельной подсказки. Если обычный ассистент предложил неправильную строку кода, разработчик может её просто не использовать. Если агент самостоятельно изменил несколько файлов или запустил операции в инфраструктуре, последствия могут быть значительно серьезнее.

Поэтому автономные агенты обычно применяются там, где можно заранее определить границы ответственности и контролировать результат.

Паттерн Chain-of-Thought при работе с задачами

При работе с ИИ часто используется идея пошагового решения сложной задачи. Вместо попытки сразу получить конечный результат система разбивает проблему на несколько этапов анализа.

Например, задача «исправить ошибку в приложении» сама по себе слишком общая. Для её решения необходимо сначала понять сообщение об ошибке, затем определить возможное место возникновения проблемы, после этого проверить предположение и только потом предложить изменение.

Такой подход помогает модели лучше работать со сложными задачами, потому что она получает возможность учитывать промежуточные этапы.

В практической разработке этот принцип проявляется в создании более структурированных запросов. Чем сложнее задача, тем важнее дать ИИ возможность учитывать контекст, ограничения и критерии результата.

При этом важно понимать, что внутренний процесс рассуждения модели не является заменой проверки человеком. Даже если ответ выглядит логичным, итоговый результат всё равно необходимо проверять на практике.

Паттерн Self-Correction

Self-correction означает способность системы анализировать собственный результат и искать возможные ошибки.

В простом сценарии модель получает задачу, формирует решение, затем дополнительно проверяет его и пытается найти проблемы.

Например, при генерации функции ИИ может самостоятельно проверить, что произойдёт при пустом входном значении, неправильном формате данных или возникновении исключения.

Этот подход особенно полезен там, где первый вариант решения может оказаться неполным. Вместо получения одного ответа система проходит дополнительный этап проверки.

Однако самопроверка не гарантирует абсолютной правильности. Если модель изначально неправильно поняла задачу, повторная проверка может выполняться уже на основе ошибочного предположения.

Поэтому self-correction повышает качество результата, но не отменяет необходимость реального выполнения программы, тестирования и проверки требований.

Паттерн Retrieval-Augmented Generation (RAG)

Одна из основных проблем при работе ИИ с кодом заключается в недостатке контекста.

Большой проект может содержать тысячи файлов, внутренние правила разработки, документацию, архитектурные решения и особенности конкретной компании. Если передать модели только небольшой фрагмент кода, она может не знать важные детали системы.

Подход RAG решает эту проблему за счёт подключения внешней информации.

Перед формированием ответа система сначала получает необходимые данные из доступного источника: документации, базы знаний, файлов проекта или других материалов. После этого эта информация используется вместе с запросом пользователя.

Например, если разработчик спрашивает, как изменить модуль авторизации, ИИ может сначала получить описание архитектуры проекта, правила безопасности и существующие реализации похожих компонентов.

Главная идея RAG заключается в том, что качество ответа зависит не только от самой модели, но и от того, какую информацию она получила перед генерацией результата.

Для разработки это особенно важно, потому что правильный ответ часто определяется не общими знаниями программирования, а конкретными правилами и особенностями текущего проекта.

Выбор подхода в зависимости от задачи

При работе с ИИ в разработке нет одного универсального режима, который подходит для всех ситуаций.

Если задача требует постоянного контроля и содержит много неизвестных условий, чаще используется подход с человеком в контуре. Если задача хорошо описана, а результат можно проверить после выполнения, подходит модель с человеком на ревью. Если процесс состоит из большого количества повторяющихся действий и имеет чёткие ограничения, можно рассматривать использование автономного агента.

Главная идея заключается в правильном распределении ответственности. ИИ может ускорять разработку, помогать анализировать информацию и выполнять часть работы, но уровень самостоятельности должен соответствовать сложности задачи и возможным последствиям ошибки.
